02-ONNX

1 ONNX (Open Neural Network Exchange)

Document Version: 1.0
Generated: December 4, 2025
Standard Version: ONNX 1.16
Status: Widely Adopted Industry Standard

1.1 Table of Contents

1.2 Overview

ONNX (Open Neural Network Exchange) is an open format designed to represent machine learning models, enabling interoperability between different frameworks. ONNX defines a common computational graph representation and a set of standard operators that can be implemented by various ML frameworks and runtime engines.

1.2.1 Key Features

- **Framework Interoperability**: Train in PyTorch, deploy in TensorFlow or vice versa
- **Hardware Optimization**: Optimized runtimes for CPUs, GPUs, and specialized accelerators
- **Versioned Operator Sets**: Backward-compatible evolution of operations
- **Extensible**: Custom operators and model metadata support
- **Binary Format**: Efficient serialization using Protocol Buffers

1.2.2 Primary Use Cases

1. **Model Export**: Export trained models from ML frameworks
2. **Cross-Framework Deployment**: Use models across different ecosystems
3. **Optimization**: Apply framework-agnostic optimizations
4. **Production Serving**: Deploy to optimized inference engines
5. **Model Sharing**: Distribute pre-trained models in a universal format

---

1.3 Authoritative References

1.3.1 Official Specifications

- **ONNX GitHub Repository**: https://github.com/onnx/onnx
- **ONNX Specification**: https://github.com/onnx/onnx/blob/main/docs/IR.md
- **Operator Schemas**: https://github.com/onnx/onnx/blob/main/docs/Operators.md
- **ONNX Model Zoo**: https://github.com/onnx/models

1.3.2 Standards Bodies

- **Linux Foundation AI & Data**: Hosts ONNX project
- **ONNX Steering Committee**: Governance and roadmap
- **Website**: https://onnx.ai/

1.3.3 Version History

- **ONNX 1.0** (2017): Initial release
- **ONNX 1.5** (2019): Added training support
- **ONNX 1.10** (2021): Enhanced operator coverage
- **ONNX 1.16** (2024): Latest stable release

1.4 Format Structure

ONNX models are serialized using **Protocol Buffers** (protobuf), a language-neutral, platform-neutral extensible mechanism for serializing structured data.

1.4.1 Core Entities

```
graph TD
    A[ModelProto] --> B[GraphProto]
    B --> C[NodeProto]
    B --> D[TensorProto]
    B --> E[ValueInfoProto]
    C --> F[AttributeProto]
    A --> G[OpSetImport]
    A --> H[MetadataProto]
```

1.4.2 1. ModelProto (Top-Level)

Role: Root container for the entire model.

Key Attributes: - `ir_version`: ONNX IR version (currently 8) - `opset_import`: Imported operator sets with versions - `producer_name`: Framework that created the model (e.g., "pytorch", "tensorflow") - `producer_version`: Version of the producing framework - `graph`: The main computational graph (GraphProto) - `metadata_props`: Key-value pairs for custom metadata

Impact: Defines compatibility and provenance of the model.

Protobuf Definition:

```
message ModelProto {
  optional int64 ir_version = 1;
  repeated OperatorSetIdProto opset_import = 8;
  optional string producer_name = 2;
  optional string producer_version = 3;
  optional string domain = 4;
  optional int64 model_version = 5;
  optional string doc_string = 6;
  optional GraphProto graph = 7;
  repeated StringStringEntryProto metadata_props = 14;
}
```

1.4.3 2. GraphProto (Computational Graph)

Role: Represents the computational graph structure.

Key Attributes: - `node`: List of computation nodes (operations) - `initializer`: Pre-initialized tensor values (weights, biases) - `input`: Graph input specifications - `output`: Graph output specifications - `value_info`: Intermediate tensor type information

Impact: Defines the actual computation flow and data transformations.

Protobuf Definition:

```
message GraphProto {
  repeated NodeProto node = 1;
  optional string name = 2;
  repeated TensorProto initializer = 5;
  optional string doc_string = 10;
  repeated ValueInfoProto input = 11;
  repeated ValueInfoProto output = 12;
  repeated ValueInfoProto value_info = 13;
}
```

1.4.4 3. NodeProto (Operator Node)

Role: Represents a single operation in the graph.

Key Attributes: - `op_type`: Operator name (e.g., "Conv", "MatMul", "Relu") - `input`: List of input tensor names - `output`: List of output tensor names - `attribute`: Operator-specific parameters - `domain`: Namespace for custom operators (empty for standard ops)

Impact: Defines individual computations and their connectivity.

Protobuf Definition:

```
message NodeProto {
  repeated string input = 1;
  repeated string output = 2;
  optional string name = 3;
  optional string op_type = 4;
  optional string domain = 7;
  repeated AttributeProto attribute = 5;
  optional string doc_string = 6;
}
```

1.4.5 4. TensorProto (Tensor Data)

Role: Stores tensor data (weights, constants).

Key Attributes: - `dims`: Shape of the tensor - `data_type`: Element data type (FLOAT, INT64, etc.) - `raw_data`: Binary blob of tensor data - `float_data`, `int64_data`, etc.: Type-specific data fields

Impact: Contains model parameters and constants.

Protobuf Definition:

```
message TensorProto {
  repeated int64 dims = 1;
  optional int32 data_type = 2;

  repeated float float_data = 4 [packed = true];
  repeated int32 int32_data = 5 [packed = true];
  repeated int64 int64_data = 7 [packed = true];
  optional bytes raw_data = 9;

  optional string name = 8;
  optional string doc_string = 12;
}
```

1.4.6 5. ValueInfoProto (Tensor Metadata)

Role: Describes tensor types and shapes.

Key Attributes: - `name`: Tensor identifier - `type`: Type information (tensor shape and element type)

Impact: Enables type checking and optimization.

1.4.7 6. AttributeProto (Operator Parameters)

Role: Stores operator-specific configuration.

Key Attributes: - `name`: Parameter name (e.g., "kernel_shape", "strides") - `type`: Attribute type (INT, FLOAT, STRING, TENSOR, etc.) - Value fields: `i`, `f`, `s`, `t`, etc. depending on type

Impact: Configures operator behavior.

1.5 Procedural Use-Cases

1.5.1 Use-Case 1: PyTorch to ONNX Export

Goal: Export a trained PyTorch model to ONNX format for deployment.

Workflow:

```
graph LR
    A[Train PyTorch Model] --> B[Define Input Spec]
    B --> C[torch.onnx.export]
    C --> D[ONNX Model File]
    D --> E[Validate with ONNX]
    E --> F[Test Inference]
    F --> G[Deploy to Runtime]
```

Step-by-Step Procedure:

1. **Train the Model**:

```
import torch
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, padding=1)
        self.relu = nn.ReLU()
        self.fc = nn.Linear(64 * 32 * 32, 10)

    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x

model = SimpleNet()
# ... training code ...
```

2. **Export to ONNX**:

```
import torch.onnx

# Prepare dummy input matching expected shape
dummy_input = torch.randn(1, 3, 32, 32)

# Export
torch.onnx.export(
    model,                      # Model to export
    dummy_input,                # Example input
    "simplenet.onnx",           # Output file
    export_params=True,         # Store trained weights
    opset_version=15,           # ONNX opset version
    do_constant_folding=True,   # Optimize constants
    input_names=['input'],      # Input tensor names
    output_names=['output'],    # Output tensor names
    dynamic_axes={              # Variable batch size
        'input': {0: 'batch_size'},
        'output': {0: 'batch_size'}
    }
)
```

3. **Validate**:

```
import onnx

# Load and check model
onnx_model = onnx.load("simplenet.onnx")
onnx.checker.check_model(onnx_model)
print("ONNX model is valid!")

# Print model info
print(onnx.helper.printable_graph(onnx_model.graph))
```

4. **Test Inference**:

```
import onnxruntime as ort
import numpy as np

# Load ONNX model
session = ort.InferenceSession("simplenet.onnx")

# Prepare input
input_data = np.random.randn(1, 3, 32, 32).astype(np.float32)

# Run inference
outputs = session.run(
    None,  # Return all outputs
    {"input": input_data}
)

print("Output shape:", outputs[0].shape)
```

1.5.2 Use-Case 2: TensorFlow to ONNX Conversion

Goal: Convert a TensorFlow SavedModel to ONNX.

Workflow:

```
graph LR
    A[TensorFlow SavedModel] --> B[tf2onnx Converter]
    B --> C[ONNX Model]
    C --> D[Optimize with ONNX Optimizer]
    D --> E[Deploy]
```

Procedure:

1. **Save TensorFlow Model**:

```
import tensorflow as tf

# Create and train model
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, 3, activation='relu', input_shape=(28, 28, 1)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(10, activation='softmax')
])

# Train model...
# model.fit(...)

# Save as SavedModel
model.save('tf_model_saved')
```

2. **Convert to ONNX**:

```
python -m tf2onnx.convert \
    --saved-model tf_model_saved \
    --output model.onnx \
    --opset 15
```

3. **Optimize**:

```
import onnx
from onnxoptimizer import optimize

# Load model
model = onnx.load("model.onnx")

# Apply optimizations
optimized_model = optimize(model, passes=[
    'eliminate_identity',
    'eliminate_nop_transpose',
    'fuse_bn_into_conv',
    'fuse_consecutive_transposes'
])

# Save optimized model
onnx.save(optimized_model, "model_optimized.onnx")
```

1.5.3 Use-Case 3: Model Optimization Pipeline

Goal: Optimize ONNX model for inference performance.

Workflow:

```
graph TD
    A[Original ONNX Model] --> B[Graph Optimization]
    B --> C[Quantization]
    C --> D[Constant Folding]
    D --> E[Optimized Model]
    E --> F{Performance Test}
    F -->|Good| G[Deploy]
    F -->|Poor| H[Adjust Parameters]
    H --> B
```

Optimization Techniques:

1. **Graph Optimization**:

```
from onnxruntime.transformers import optimizer

# Optimize transformer models
optimized_model = optimizer.optimize_model(
    "model.onnx",
    model_type='bert',
    num_heads=12,
    hidden_size=768
)
optimized_model.save_model_to_file("model_optimized.onnx")
```

2. **Quantization (INT8)**:

```
import onnxruntime.quantization as quantization

# Dynamic quantization
quantization.quantize_dynamic(
    "model.onnx",
    "model_quantized.onnx",
    weight_type=quantization.QuantType.QInt8
)

# Static quantization (requires calibration data)
def calibration_data_reader():
    # Yield calibration batches
    for i in range(100):
        yield {"input": np.random.randn(1, 3, 224, 224).astype(np.float32)}

quantization.quantize_static(
    "model.onnx",
    "model_int8.onnx",
    calibration_data_reader()
)
```

1.6 ONNX Operators

ONNX defines a comprehensive set of standard operators covering various neural network operations.

1.6.1 Operator Categories

Category	Operators	Use Cases
Neural Network	Conv, MaxPool, BatchNormalization, Dropout	CNN layers
Math	Add, Sub, Mul, Div, MatMul, Gemm	Arithmetic operations
Activation	Relu, Sigmoid, Tanh, Softmax, Gelu	Non-linearities
Reduction	ReduceMean, ReduceSum, ArgMax, ArgMin	Aggregations
Shape Manipulation	Reshape, Transpose, Concat, Split	Tensor transformations

Category	Operators	Use Cases
Logical	And, Or, Not, Equal, Greater	Boolean operations
RNN	LSTM, GRU, RNN	Recurrent networks
Object Detection	NMS, RoiAlign	Vision tasks

1.6.2 Key Operators in Detail

1.6.2.1 Conv (Convolution)

Purpose: 2D convolutional operation for image processing.

Attributes: - `kernel_shape`: Filter dimensions [height, width] - `strides`: Step size [stride_h, stride_w] - `pads`: Padding [top, left, bottom, right] - `dilations`: Dilation factors - `group`: Number of groups (for grouped convolutions)

Inputs: 1. `X`: Input tensor [N, C_in, H, W] 2. `W`: Weight tensor [C_out, C_in/group, kernel_h, kernel_w] 3. `B`: Optional bias [C_out]

Outputs: 1. `Y`: Output tensor [N, C_out, H_out, W_out]

Example Node:

```
conv_node = onnx.helper.make_node(
    'Conv',
    inputs=['X', 'W', 'B'],
    outputs=['Y'],
    kernel_shape=[3, 3],
    pads=[1, 1, 1, 1],
    strides=[1, 1]
)
```

1.6.2.2 MatMul (Matrix Multiplication)

Purpose: General matrix multiplication.

Inputs: 1. `A`: Matrix [M, K] 2. `B`: Matrix [K, N]

Outputs: 1. `Y`: Result [M, N]

Example:

```
matmul_node = onnx.helper.make_node(
    'MatMul',
    inputs=['A', 'B'],
    outputs=['Y']
)
```

1.6.2.3 Softmax

Purpose: Softmax activation for classification.

Attributes: - `axis`: Dimension to apply softmax (default: -1)

Formula: $\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$

Example:

```
softmax_node = onnx.helper.make_node(
    'Softmax',
    inputs=['X'],
    outputs=['Y'],
    axis=1
)
```

1.7 Examples

1.7.1 Example 1: Complete ONNX Model (Pseudo-Binary Structure)

Logical Structure (for binary ONNX file):

```
ModelProto {
  ir_version: 8
  producer_name: "pytorch"
  producer_version: "2.0.0"
  opset_import {
    domain: ""
    version: 15
  }
  graph: GraphProto {
    name: "simple_classifier"
    node: [
      NodeProto {
        input: ["input", "conv_weight", "conv_bias"]
        output: ["conv_output"]
        op_type: "Conv"
        attribute: [
          {name: "kernel_shape", ints: [3, 3]},
          {name: "strides", ints: [1, 1]}
        ]
      },
      NodeProto {
        input: ["conv_output"]
        output: ["relu_output"]
        op_type: "Relu"
      },
      NodeProto {
        input: ["relu_output", "fc_weight", "fc_bias"]
        output: ["logits"]
        op_type: "Gemm"
      },
      NodeProto {
        input: ["logits"]
        output: ["output"]
        op_type: "Softmax"
        attribute: [{name: "axis", i: 1}]
      }
    ]
    initializer: [
      TensorProto {
        name: "conv_weight"
        dims: [64, 3, 3, 3]
        data_type: FLOAT
        raw_data: <binary blob>
      },
      TensorProto {
        name: "conv_bias"
        dims: [64]
        data_type: FLOAT
        raw_data: <binary blob>
      },
      TensorProto {
        name: "fc_weight"
        dims: [10, 64]
        data_type: FLOAT
```

```
      data_type: FLOAT
      raw_data: <binary blob>
    },
    TensorProto {
      name: "fc_bias"
      dims: [10]
      data_type: FLOAT
      raw_data: <binary blob>
    }
  ]
  input: [
    ValueInfoProto {
      name: "input"
      type: {
        tensor_type: {
          elem_type: FLOAT
          shape: {dim: [{dim_param: "batch"}, {dim_value: 3}, {dim_value: 32}, {dim_value: 32}]}
        }
      }
    }
  ]
  output: [
    ValueInfoProto {
      name: "output"
      type: {
        tensor_type: {
          elem_type: FLOAT
          shape: {dim: [{dim_param: "batch"}, {dim_value: 10}]}
        }
      }
    }
  ]
 }
}
```

1.7.2 Example 2: Creating ONNX Model Programmatically

Python Code (actual implementation):

```python
import onnx
from onnx import helper, TensorProto
import numpy as np

# Create input/output value info
input_info = helper.make_tensor_value_info(
    'input', TensorProto.FLOAT, [1, 3, 224, 224]
)
output_info = helper.make_tensor_value_info(
    'output', TensorProto.FLOAT, [1, 1000]
)

# Create weight tensors
conv_weight = np.random.randn(64, 3, 7, 7).astype(np.float32)
conv_weight_tensor = helper.make_tensor(
    'conv_weight',
    TensorProto.FLOAT,
    conv_weight.shape,
    conv_weight.flatten()
)

fc_weight = np.random.randn(1000, 64).astype(np.float32)
fc_weight_tensor = helper.make_tensor(
    'fc_weight',
    TensorProto.FLOAT,
    fc_weight.shape,
    fc_weight.flatten()
)

# Create nodes
conv_node = helper.make_node(
```

```
    'Conv',
    inputs=['input', 'conv_weight'],
    outputs=['conv_output'],
    kernel_shape=[7, 7],
    strides=[2, 2],
    pads=[3, 3, 3, 3]
)

relu_node = helper.make_node(
    'Relu',
    inputs=['conv_output'],
    outputs=['relu_output']
)

pool_node = helper.make_node(
    'GlobalAveragePool',
    inputs=['relu_output'],
    outputs=['pool_output']
)

flatten_node = helper.make_node(
    'Flatten',
    inputs=['pool_output'],
    outputs=['flatten_output']
)

fc_node = helper.make_node(
    'MatMul',
    inputs=['flatten_output', 'fc_weight'],
    outputs=['output']
)

# Create graph
graph = helper.make_graph(
    nodes=[conv_node, relu_node, pool_node, flatten_node, fc_node],
    name='simple_cnn',
    inputs=[input_info],
    outputs=[output_info],
    initializer=[conv_weight_tensor, fc_weight_tensor]
)

# Create model
model = helper.make_model(graph, producer_name='custom_builder')
model.opset_import[0].version = 15

# Check and save
onnx.checker.check_model(model)
onnx.save(model, 'custom_model.onnx')
print("Model created successfully!")
```

1.7.3 Example 3: Model Inspection

Inspecting ONNX Model (actual code):

```python
import onnx

# Load model
model = onnx.load("model.onnx")

# Print basic info
print(f"IR Version: {model.ir_version}")
print(f"Producer: {model.producer_name} {model.producer_version}")
print(f"Opset Version: {model.opset_import[0].version}")

# Graph info
graph = model.graph
print(f"\nGraph Name: {graph.name}")
print(f"Number of Nodes: {len(graph.node)}")
print(f"Number of Initializers: {len(graph.initializer)}")

# Input/Output shapes
print("\nInputs:")
for input_tensor in graph.input:
    print(f"  {input_tensor.name}: {[d.dim_value or d.dim_param for d in input_tensor.type.tensor_type.shape.dim]}")

print("\nOutputs:")
for output_tensor in graph.output:
    print(f"  {output_tensor.name}: {[d.dim_value or d.dim_param for d in output_tensor.type.tensor_type.shape.dim]}")

# Node details
print("\nNodes:")
for node in graph.node:
    print(f"  {node.op_type}: {node.input} -> {node.output}")
    if node.attribute:
        for attr in node.attribute:
            print(f"    {attr.name}: {onnx.helper.get_attribute_value(attr)}")
```

1.7.4 Example 4: ONNX Runtime Inference (Production)

Optimized Inference (actual code):

```python
import onnxruntime as ort
import numpy as np

class ONNXInferenceEngine:
    def __init__(self, model_path, use_gpu=False):
        # Set execution providers
        providers = ['CUDAExecutionProvider', 'CPUExecutionProvider'] if use_gpu else ['CPUExecutionProvider']

        # Create session with options
        sess_options = ort.SessionOptions()
        sess_options.graph_optimization_level = ort.GraphOptimizationLevel.ORT_ENABLE_ALL
        sess_options.intra_op_num_threads = 4

        self.session = ort.InferenceSession(
            model_path,
            sess_options=sess_options,
            providers=providers
        )

        # Get input/output metadata
        self.input_name = self.session.get_inputs()[0].name
        self.output_name = self.session.get_outputs()[0].name
        self.input_shape = self.session.get_inputs()[0].shape

    def predict(self, input_data):
        """Run inference on input data"""
        # Ensure correct dtype
        if not isinstance(input_data, np.ndarray):
            input_data = np.array(input_data)
        if input_data.dtype != np.float32:
            input_data = input_data.astype(np.float32)

        # Run inference
        outputs = self.session.run(
            [self.output_name],
            {self.input_name: input_data}
        )

        return outputs[0]

    def batch_predict(self, batch_data, batch_size=32):
        """Efficient batch inference"""
        results = []
        for i in range(0, len(batch_data), batch_size):
            batch = batch_data[i:i+batch_size]
            batch_array = np.array(batch, dtype=np.float32)
            output = self.predict(batch_array)
            results.append(output)
        return np.concatenate(results, axis=0)

# Usage
engine = ONNXInferenceEngine("model.onnx", use_gpu=True)

# Single inference
input_data = np.random.randn(1, 3, 224, 224).astype(np.float32)
prediction = engine.predict(input_data)
print(f"Prediction shape: {prediction.shape}")

# Batch inference
batch_data = [np.random.randn(3, 224, 224) for _ in range(100)]
batch_predictions = engine.batch_predict(batch_data, batch_size=16)
print(f"Batch predictions shape: {batch_predictions.shape}")
```

1.8 Tools & Ecosystem

1.8.1 Model Conversion Tools

Tool	Description	Homepage
PyTorch	Native ONNX export via torch.onnx	https://pytorch.org/docs/stable/onnx.html
TensorFlow (tf2onnx)	TF/Keras to ONNX converter	https://github.com/onnx/tensorflow-onnx
ONNX-TensorFlow	Bi-directional TF↔ONNX	https://github.com/onnx/onnx-tensorflow
sklearn-onnx	Scikit-learn to ONNX	https://github.com/onnx/sklearn-onnx
CoreML Tools	CoreML↔ONNX conversion	https://github.com/apple/coremltools
Keras2onnx	Keras to ONNX	https://github.com/onnx/keras-onnx

1.8.2 Runtime & Inference

Tool	Description	Homepage
ONNX Runtime	Microsoft's cross-platform inference engine	https://onnxruntime.ai/
TensorRT	NVIDIA's high-performance inference	https://developer.nvidia.com/tensorrt
OpenVINO	Intel's inference toolkit	https://docs.openvino.ai/
ONNX.js	Browser-based inference	https://github.com/microsoft/onnxjs
MXNet	Apache MXNet with ONNX support	https://mxnet.apache.org/

1.8.3 Optimization & Validation

Tool	Description	Homepage
ONNX Optimizer	Graph optimization passes	https://github.com/onnx/optimizer
ONNX Simplifier	Simplify ONNX models	https://github.com/daquexian/onnx-simplifier
Netron	Neural network visualizer	https://netron.app/
ONNX Checker	Model validation (built-in)	Part of ONNX package

1.8.4 Model Repositories

Repository	Description	Homepage
ONNX Model Zoo	Official pre-trained models	https://github.com/onnx/models
Hugging Face Hub	Community models (many ONNX)	https://huggingface.co/models
ONNX Model Hub	Curated model collection	https://onnx.ai/models

---

1.9 Best Practices

1.9.1 1. Version Management

```
# Always specify opset version explicitly
torch.onnx.export(
    model,
    dummy_input,
    "model.onnx",
    opset_version=15  # Use latest stable opset
)
```

1.9.2 2. Dynamic Shapes

```
# Enable dynamic batch size for flexibility
dynamic_axes = {
    'input': {0: 'batch_size'},
    'output': {0: 'batch_size'}
}

torch.onnx.export(
    model, dummy_input, "model.onnx",
    dynamic_axes=dynamic_axes
)
```

1.9.3 3. Model Validation

```
import onnx

# Always validate after export
model = onnx.load("model.onnx")
onnx.checker.check_model(model)

# Check for potential issues
from onnx import shape_inference
inferred_model = shape_inference.infer_shapes(model)
```

1.9.4 4. Performance Testing

```
import time
import numpy as np

# Benchmark inference
def benchmark(session, input_data, num_runs=100):
    # Warmup
    for _ in range(10):
        session.run(None, {session.get_inputs()[0].name: input_data})

    # Measure
    start = time.time()
    for _ in range(num_runs):
        session.run(None, {session.get_inputs()[0].name: input_data})
    end = time.time()

    avg_time = (end - start) / num_runs * 1000  # ms
    print(f"Average inference time: {avg_time:.2f} ms")

session = ort.InferenceSession("model.onnx")
input_data = np.random.randn(1, 3, 224, 224).astype(np.float32)
benchmark(session, input_data)
```

1.9.5 5. Metadata Management

```
# Add model metadata
model.metadata_props.append(
    onnx.StringStringEntryProto(key="description", value="Image classifier")
)
model.metadata_props.append(
    onnx.StringStringEntryProto(key="author", value="ML Team")
)
model.metadata_props.append(
    onnx.StringStringEntryProto(key="license", value="Apache-2.0")
)
```

1.10 References

1.10.1 Official Documentation

- ONNX GitHub: https://github.com/onnx/onnx
- ONNX Tutorials: https://github.com/onnx/tutorials
- ONNX Runtime Docs: https://onnxruntime.ai/docs/

1.10.2 Related Standards

- **Protocol Buffers**: https://protobuf.dev/
- **Model Cards**: See 04-Model-Cards.md
- **Vector Embeddings**: See A01-Vector-Embeddings-Standards.md

1.10.3 Community Resources

- ONNX Slack: https://lfaifoundation.slack.com/
- ONNX Working Groups: https://github.com/onnx/onnx/blob/main/community/readme.md

---

Navigation: Back to Index | Previous: Neural Networks Fundamentals | Next: OpenAPI for AI Services